

Task-Anchored Representation Shaping for Pre-Trained Model-Based Continual Learning

Zhiming Xu^{1,2}, Huiyu Yi^{1,2}, Zhen-Hao Xie^{1,2}, Baile Xu^{1,2}, Furao Shen^{1,2,†}, Jian Zhao⁴, Suorong Yang^{1,3}

Abstract

Pre-trained models (PTMs) provide a strong foundation for continual learning by offering stable representations that facilitate lightweight adaptation to new tasks. However, adapting well to each task does not ensure reliable inference over all learned tasks. Since task boundaries are often artificial and semantically entangled, an input from an unknown task can remain ambiguous even with strong PTM features, making cross-task prediction a key bottleneck. We propose Task-Anchored Inference Latent Shaping (TAILS), a lightweight post-PTM module that can be integrated into diverse continual learners and optimized through a decoupled step. TAILS uses fixed task anchors as persistent references to accumulated knowledge. It interprets each sample’s feature representation relative to these references, then composes relevant evidence across tasks into latent recall. Rather than selecting a task-specific path or adjusting classifier outputs, TAILS uses latent recall to directly correct the feature representation before prediction. It therefore resolves cross-task ambiguity at the representation level, while leaving the original PTM, method-specific modules, and classifier unchanged. Extensive experiments across multiple PTM-based continual learning paradigms show that TAILS can improve classification and task-inference performance with modest parameter overhead and negligible inference cost.

1 Introduction

Modern deep models are increasingly deployed in non-stationary environments, where data arrive sequentially, and their distributions evolve over time (Gomes et al. 2017; Ye et al. 2019; Chen et al. 2021, 2022; Yang, Shen, and Zhao 2024). To remain effective in such environments, models must continually incorporate new knowledge without erasing what has already been learned, which is the central objective of continual learning (CL) (Liang and Li 2024; Yang et al. 2025). However, sequential updates can cause catastrophic forgetting (De Lange et al. 2021), whereby learning new data

¹National Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China. ²School of Artificial Intelligence, Nanjing University, Nanjing, China. ³School of Computer Science, Nanjing University, Nanjing, China. ⁴School of Electronic Science and Engineering, Nanjing University, China.

[†]Correspondence to: frshen@nju.edu.cn

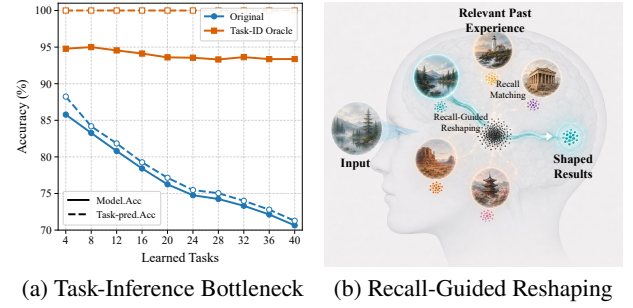


Figure 1: Motivation of TAILS. **Left:** Accuracy under the original and task-ID-oracle settings throughout continual learning. **Right:** For an ambiguous input, TAILS interprets its representation against accumulated task structures, composes sample-specific latent recall, and uses it to condition a feature correction before prediction by the original classifier.

overwrites previously acquired knowledge. Although conventional CL methods alleviate this problem (Zheng et al. 2025; Aghasanli, Li, and Angelov 2025; Li et al. 2025; Nori, Kim, and Wang 2025), they still learn representations primarily from limited incremental, task-specific data, restricting their ability to maintain a stable and discriminative feature space across tasks. Pre-trained models (PTMs) offer a stronger foundation for CL by supplying representations learned from large-scale data (Wang et al. 2022a; Zhou et al. 2025a; Wang, Zhou, and Ye 2025; Sun et al. 2025). Rather than learning representations from scratch on scarce incremental data, PTM-based methods adapt to incoming tasks through lightweight components while keeping the backbone frozen or lightly updated, yielding stable and discriminative features throughout continual learning.

Despite these advantages, data arriving at each learning stage are conventionally organized as distinct tasks (Zhou et al. 2024a), whereas inference must be performed over all accumulated tasks without task identity. As powerful PTMs and task-wise adaptation have substantially reduced intra-task prediction errors, the primary challenge has shifted from learning individual tasks to identifying and exploiting relevant cross-task knowledge, making this identification and use increasingly important. For example, in continual

classification (Zhou et al. 2024c), classes are typically assigned to tasks in a random order so that the resulting task boundaries may exhibit little meaningful semantic separation. Consequently, inputs from different tasks may remain highly overlapping in the feature space, making task inference inherently ambiguous. This ambiguity becomes increasingly severe as more tasks are accumulated, since the learner must distinguish among a growing number of semantically entangled task distributions. Existing methods mainly address this problem through task selection (Wang et al. 2022a; Xu et al. 2025; Sun et al. 2025) or model ensembling (Zhou et al. 2024b; Wang, Zhou, and Ye 2025). These strategies organize task-specific components through routing or aggregation, but their effectiveness depends heavily on accurately estimating task relevance. When task boundaries lack clear semantic separation, inaccurate task inference inevitably leads to selecting or weighting inappropriate task-specific components, while leaving the underlying feature ambiguity unresolved. Fig. 1a illustrates that under a Task-ID Oracle setting, where ground-truth task identity is supplied at inference, accuracy for existing PTM-based CL methods remains nearly flat as tasks accumulate; without this oracle, task-prediction accuracy drops sharply and converges with overall accuracy in later tasks. This convergence suggests that existing methods already perform reliably once the correct task is identified, whereas errors in task inference increasingly dominate final prediction performance, making task inference a major bottleneck for continual learning. This naturally raises a new and pressing question for PTM-based CL: *how can a PTM-based CL learner effectively acquire and preserve task-level structures for reliable cross-task inference?*

To address the cross-task inference bottleneck, we internalize accumulated task structures into the current representation before prediction. Specifically, we propose Task-Anchored Inference Latent Shaping (TAILS), a lightweight plug-and-play module inserted after the PTM feature encoder and before the classifier. Unlike selection and ensembling, TAILS does not route samples to a discrete task-specific component; instead, it composes a soft, input-conditioned summary of evidence across all learned tasks and uses it to correct the feature itself, leaving the original classifier and prediction mechanism unchanged. Concretely, given a PTM-derived feature, TAILS forms a compact query, matches it against task slots built from accumulated task anchors, composes the retrieved evidence into a sample-specific latent recall, and uses this recall to condition a correction of the feature before classification (Fig. 1b). TAILS is shared across all learned tasks and trained in a decoupled step after the base learner’s standard training completes, so it augments inference without altering or risking the base learner’s original training pipeline. To keep this recall process stable as new tasks arrive, we construct fixed task anchors from class prototypes and support features, which serve as compact, persistent references both for preserving the recall space during optimization and for retrieving accumulated evidence at inference. Because TAILS is a plug-in, post-hoc module rather than a replacement for any existing component, it can be attached to a wide range of PTM-based continual learners with modest parameter overhead and negligible inference cost. It

consistently elevates diverse state-of-the-art baselines such as TUNA and CL-LoRA by up to 5.05 percentage points while incurring as little as 3.0% additional storage overhead when added to storage-heavy base learners, highlighting both its efficiency and effectiveness. The main contributions are summarized as follows: (1) We propose TAILS, a plug-in post-PTM module that performs assisted inference with task-anchored recall, enabling base learners to shape representations without modifying their original prediction mechanisms. (2) We construct task anchors from class prototypes and support features. These anchors provide compact references for preserving and accessing accumulated task structures during optimization and inference. (3) We conduct extensive experiments across diverse PTM-based CL paradigms, showing that TAILS consistently improves performance with modest parameter overhead and negligible inference cost.

2 Related Work

2.1 Continual Learning

Continual learning (CL) aims to continuously acquire knowledge from non-stationary data streams while preserving previously learned capabilities. A broad body of work has addressed this objective across diverse continual settings through complementary mechanisms (Zhu et al. 2025; Zhang et al. 2025; Zhou et al. 2025b; Liu et al. 2025; Gao, Zhao, and Fu 2026). Replay-based methods retain past information by storing representative samples or reconstructing compact historical distributions from prototypes and features (Zhu et al. 2025; Zhang et al. 2025). Regularization- and distillation-based approaches constrain parameter updates or model responses to remain consistent with previously learned functions (Gao et al. 2025; Zang et al. 2026). Prototype- and feature-based methods preserve class-level statistics, refine historical prototypes, or compensate for representation drift across increments (Zhu et al. 2025; Gao, Zhao, and Fu 2026). Architecture- and ensemble-based methods instead isolate or combine knowledge by expanding model capacity, organizing related classes, or aggregating multiple model states (Lai et al. 2025; Lee, Hayat, and Yun 2025). However, these methods still construct and maintain their knowledge from the limited incremental stream, which constrains the quality and generalizability of the learned feature space. This limitation has motivated the growing adoption of large-scale pre-trained models as a stronger representational foundation for continual learning.

2.2 Pre-Trained Model-based CL

Recent studies have increasingly incorporated pre-trained models into continual learning. Early approaches adapt frozen PTMs with learnable prompts, including L2P (Wang et al. 2022b), DualPrompt (Wang et al. 2022a), CODA-Prompt (Smith et al. 2023), and APT (Chen et al. 2025). Adapter-based methods insert lightweight modules for efficient task adaptation. EASE (Zhou et al. 2024b) builds task-specific adapter subspaces and ensembles their representations, while MOS (Sun et al. 2025) trains task-specific adapters with improved retrieval to reduce forgetting.

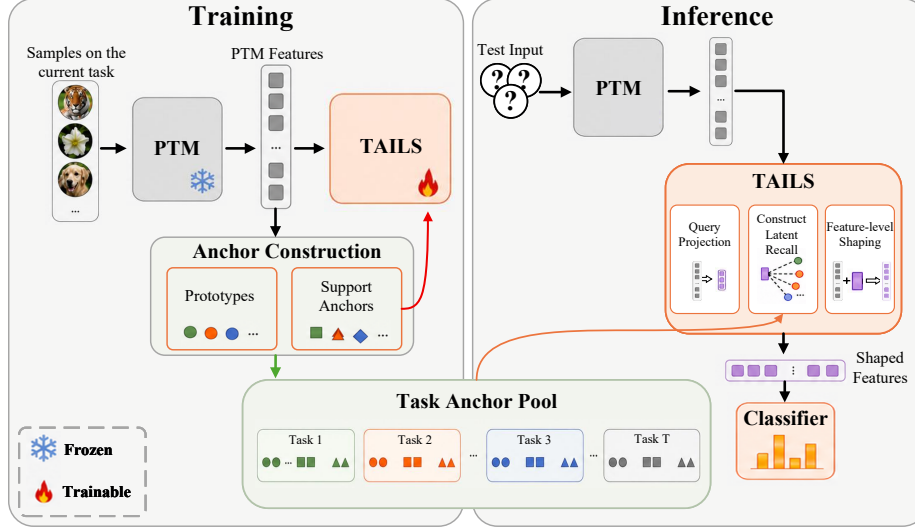


Figure 2: Illustration of TAILS. After each task is learned by the base learner, its class prototypes and features are retained as task anchors. During continual optimization, the accumulated anchor pool maintains a coherent latent-recall space. At inference, a query derived from the current representation addresses the accumulated task structures and composes sample-specific latent recall, which conditions feature shaping before prediction by the original classifier.

SEMA (Wang et al. 2025) dynamically expands modularized adapters under distribution shifts, whereas TUNA (Wang, Zhou, and Ye 2025) combines task-specific and universal adapters for specialization and knowledge sharing. In addition, MOS and TUNA preserve class-alignment statistics from previous tasks, which can be viewed as a form of non-sample replay. Such retained statistics mainly support adapter retrieval and classifier calibration, rather than directly resolving sample-specific ambiguity at the representation level before prediction. LoRA-based methods perform adaptation through low-rank updates. SD-LoRA (Wu et al. 2025) decouples LoRA direction and magnitude, while CL-LoRA (He, Duan, and Zhu 2025) combines task-shared and task-specific low-rank adapters. Other works revisit PTM-based CIL through prototype learning (Zhou et al. 2025a) and analytic classifiers (Yi et al. 2026), further demonstrating the benefits of strong PTM representations for continual recognition.

3 Preliminaries

A continual learning (CL) model learns from a sequence of evolving data streams $D_1, D_2, \dots, D_t$, where each stream $D_i = \{(\mathbf{x}_j, y_j)\}_{j=1}^{n_i}$ contains n_i training samples. For any stage t , the current stream D_t is treated as a new task, with its label space denoted by Y_t . The model has encountered the data streams $D_{1:t} = D_1 \cup D_2 \cup \dots \cup D_t$ and the accumulated label space $\mathcal{Y}_{1:t} = Y_1 \cup Y_2 \cup \dots \cup Y_t$. The goal is to learn a CL model $f_t : \mathcal{X} \rightarrow \mathcal{Y}_{1:t}$ that performs well on all learned data. The performance of f_t is evaluated by its classification error on the accumulated test set:

$$\text{Err}(f_t) = \mathbb{E}_{(\mathbf{x}, y) \in D_{1:t}^{\text{test}}} [\mathbb{I}(f_t(\mathbf{x}) \neq y)], \quad (1)$$

where $\mathbb{I}(\cdot)$ is the indicator function. We follow the commonly adopted class-incremental continual learning setting, where

task IDs are unavailable at inference. The model must make correct predictions among all classes using only the sample.

This paper focuses on PTM-based CL, where the learner adopts a powerful pre-trained encoder as its feature extractor, therefore providing a stronger foundation for continual adaptation. Given an input sample $\mathbf{x}$, a PTM-based learner first extracts its representation through the pre-trained encoder ϕ_{ptm} and then predicts the label with a classifier G :

$$\mathbf{h} = \phi_{\text{ptm}}(\mathbf{x}), \quad \hat{y} = G(\mathbf{h}). \quad (2)$$

To better adapt the PTM to downstream continual tasks, recent PTM-based CL methods usually freeze the PTM backbone and introduce lightweight fine-tuning modules. The PTM-based learner is trained only on the current task data D_t at task t ; once it proceeds to the next task, previous training images are inaccessible.

4 Method

TAILS introduces *task-anchored assisted inference* at the feature level. Unlike task routing, which selects a task-specific path, or output calibration, which adjusts logit scores from the classifier, TAILS performs auxiliary inference directly within the feature space. Specifically, a shared post-PTM module uses the retrieved latent recall to reposition an ambiguous input in the accumulated class space, while leaving the base learner and its original prediction mechanism unchanged, as illustrated in Fig. 2. In the following, we describe the TAILS module, its training objective, and its integration with different PTM-based learners.

4.1 Task-Anchored Latent Recall and Shaping

TAILS is a unified post-PTM module $\mathcal{F}_{\text{TAILS}}(\mathbf{h}; \mathcal{A})$ shared across all learning tasks. It interprets the current representation against task structures retained in the accumulated

task anchors, composes relevant historical evidence into latent recall, and internalizes the recalled evidence as a feature correction. After the base learner finishes task i , we construct an anchor subset $\mathcal{A}_i$ from its post-update sample features. For each class, we retain its class prototype and uniformly sample M support features without replacement, resulting in a class-balanced anchor set. If fewer than M samples are available, all features of that class are retained. The prototypes capture class-level semantics, while the support anchors preserve local feature variations. Once constructed, each $\mathcal{A}_i$ remains fixed, and the accumulated task anchor pool $\mathcal{A} = \{\mathcal{A}_1, \dots, \mathcal{A}_t\}$ provides a persistent reference for cross-task optimization and inference. Since continual tasks may overlap semantically, a representation that is discriminative within its local task can remain ambiguous among classes learned at different tasks. TAILS addresses this ambiguity through soft, input-dependent access to the accumulated anchor structures. To establish this access, TAILS represents the current input and each task-specific anchor structure in the same compact space. Given a representation $\mathbf{h} \in \mathbb{R}^d$ extracted from the input sample, we first derive a compact normalized query $\mathbf{q} \in \mathbb{R}^{d_q}$, where $d_q = 64$:

$$\mathbf{q} = \text{Norm}(\text{LN}(W_q \mathbf{h})), \quad (3)$$

where $\text{LN}(\cdot)$ denotes layer normalization (Ba, Kiros, and Hinton 2016), and $\text{Norm}(\cdot)$ denotes ℓ_2 normalization. The query provides an input-conditioned address for accessing the accumulated anchor structures. Let $\mathbf{A}_i \in \mathbb{R}^{|\mathcal{A}_i| \times d}$ denote the matrix obtained by stacking all anchors in $\mathcal{A}_i$. To make the stored anchors directly comparable with the query $\mathbf{q}$, TAILS projects every anchor of task i from the original feature space into the same d_q -dimensional query space:

$$\bar{\mathbf{A}}_i = \text{Norm}(\text{LN}(\mathbf{A}_i W_a^T)). \quad (4)$$

Then TAILS summarizes the embedded anchor structure into K task slots:

$$\mathbf{Z}_i = \text{Norm} \left[\text{softmax} \left(\frac{\mathbf{S} \bar{\mathbf{A}}_i^T}{\sqrt{d_q}} \right) \bar{\mathbf{A}}_i \right], \quad (5)$$

where the softmax is applied row-wise over the anchors, $\mathbf{S} \in \mathbb{R}^{K \times d_q}$ contains K learnable slot vectors shared across tasks, and $\mathbf{Z}_i = [\mathbf{z}_{i,1}, \dots, \mathbf{z}_{i,K}]^T$ contains the task slots derived from $\mathcal{A}_i$. Thus, all tasks share the same slot construction, while the content of $\mathbf{Z}_i$ is determined by the corresponding anchor set $\mathcal{A}_i$. The normalized mean of the task slots defines a coarse address for task i :

$$\mathbf{T}_c^i = \text{Norm} \left(\frac{1}{K} \sum_{k=1}^K \mathbf{z}_{i,k} \right). \quad (6)$$

Given a sample query $\mathbf{q}$, TAILS first evaluates the relevance of each accumulated task structure and then the relevance of its individual task slots:

$$\pi_i(\mathbf{q}) = \text{softmax}_i(\mathbf{q}^T \mathbf{T}_c^i), \quad (7)$$

$$\rho_{i,k}(\mathbf{q}) = \text{softmax}_k(\mathbf{q}^T \mathbf{z}_{i,k}). \quad (8)$$

$$\mathbf{r} = \sum_{i=1}^t \pi_i(\mathbf{q}) \sum_{k=1}^K \rho_{i,k}(\mathbf{q}) \mathbf{z}_{i,k}. \quad (9)$$

We refer to $\mathbf{r}$ as *latent recall*: an input-conditioned summary of accumulated feature structures that provides auxiliary evidence for interpreting the current representation. The task organization acts only as an indexing scaffold rather than a hard task decision. Since $\pi_i(\mathbf{q})$ is soft, latent recall may combine relevant task slots from multiple tasks when semantic neighborhoods cross the procedural boundaries of continual training. The resulting $\mathbf{r}$ provides the historical context that TAILS internalizes through a conditioned feature displacement. Specifically, the current feature determines a candidate correction direction, while latent recall modulates which dimensions of that correction are expressed:

$$\mathbf{h}' = \mathcal{F}_{\text{TAILS}}(\mathbf{h}; \mathcal{A}) = \text{Norm}(\text{LN}(W_b \mathbf{h}) + \Delta \mathbf{h}(\mathbf{h}, \mathbf{r})), \quad (10)$$

$$\Delta \mathbf{h}(\mathbf{h}, \mathbf{r}) = W_o \text{GELU}(W_h \text{LN}(\mathbf{h}) \odot \sigma(W_r \text{LN}(\mathbf{r}))). \quad (11)$$

Here, W_b is an identity-initialized linear mapping, allowing this branch to start from the original PTM feature and avoid disturbing pre-trained semantics at the beginning of TAILS training (Houlsby et al. 2019). Unlike conventional residual feature adaptation conditioned only on the current representation, TAILS conditions its correction on latent recall derived from accumulated task structures.

4.2 Latent-Recall-Preserving Optimization

The TAILS module is incrementally updated after each task. Once the base learner completes task t , we freeze its representation learner and classifier and optimize only TAILS in the feature space. Let $\mathcal{F}_t^{\text{cur}}$ denote the set of post-update representations extracted from the current-task samples. The optimization uses $\mathcal{F}_t^{\text{cur}}$ together with the historical anchor set $\mathcal{A}_{1:t-1} = \bigcup_{i=1}^{t-1} \mathcal{A}_i$. These two sources play complementary roles: current-task representations incorporate newly acquired structures into the shared TAILS mapping, whereas historical anchors provide stable reference features for preserving previously established classifier compatibility, recall addresses, and local feature geometry without retaining any previous training samples. Since the anchors remain fixed and the base learner is never revisited, this procedure updates only how TAILS interprets the accumulated feature space without modifying the original continual-learning model. Accordingly, each training feature $\mathbf{h}_{\text{train}} \in \mathcal{F}_t^{\text{cur}} \cup \mathcal{A}_{1:t-1}$ is shaped as $\mathbf{h}'_{\text{train}} = \mathcal{F}_{\text{TAILS}}(\mathbf{h}_{\text{train}}; \mathcal{A})$, and its corresponding query $\mathbf{q}_{\text{train}}$ is computed from $\mathbf{h}_{\text{train}}$ using Eq. (3). We impose three complementary constraints:

$$\mathcal{L}_{\text{TAILS}} = \mathcal{L}_{\text{cls}} + \mathcal{L}_{\text{task}} + \mathcal{L}_{\text{stab}}. \quad (12)$$

The classifier-compatibility constraint ensures that the shaped feature remains meaningful under the frozen decision rule of base learner $\mathcal{B}$:

$$\mathcal{L}_{\text{cls}} = \text{CE}(\mathcal{G}_{\mathcal{B}}(\mathbf{h}'_{\text{train}}), y). \quad (13)$$

Gradients pass through the fixed classifier only to TAILS. Thus, the objective does not learn a new classifier; it teaches latent recall to produce feature corrections that are compatible with the base learner’s established semantic space.

The recall-addressing constraint organizes where the current query should seek evidence. Let $\tau(y)$ denote the task

ID associated with the class y . We supervise the similarities between the query and accumulated task centers:

$$\mathcal{L}_{\text{task}} = \text{CE}\left(\left\{\text{sim}_{\cos}(\mathbf{q}_{\text{train}}, \mathbf{T}_c^i)\right\}_{i=1}^t, \tau(y)\right). \quad (14)$$

This signal learns the addressing behavior of latent recall without requiring a task ID at inference. The supervised center response establishes a reliable address, whereas Eq. (9) still performs soft composition of task slots for the actual feature correction.

The semantic-preservation constraint limits excessive deviation from the original PTM representation:

$$\mathcal{L}_{\text{stab}} = 1 - \frac{\mathbf{h}'_{\text{train}} \top \mathbf{h}_{\text{train}}}{\|\mathbf{h}'_{\text{train}}\|_2 \|\mathbf{h}_{\text{train}}\|_2}. \quad (15)$$

Together, the three terms preserve classifier compatibility, recall-address consistency, and continuity of PTM semantics, allowing TAILS to incorporate new task structures while maintaining a coherent latent-recall across all tasks.

4.3 Integrating TAILS with PTM Learners

TAILS operates at the representation interface shared by otherwise heterogeneous PTM-based learners. This placement allows the same unified module to assist inference while preserving each base learner’s adaptation mechanism and original decision rule.

For methods that ultimately produce a single representation for prediction, including raw PTM, prompt-based methods, and adapter-based methods that fuse multiple adapter outputs into one (Wang, Zhou, and Ye 2025), TAILS applies $\mathcal{F}_{\text{TAILS}}(\cdot; \mathcal{A})$ to the resulting feature $\mathbf{h}$ and feeds the shaped representation $\mathbf{h}'$ from Eq. (10) to the original classifier. For methods that expose task-specific candidate features before prediction, such as task-specific adapters (Sun et al. 2025) or task-specific LoRA branches (He, Duan, and Zhu 2025), the same TAILS module is applied independently to each candidate feature:

$$\mathbf{h}'^i = \mathcal{F}_{\text{TAILS}}(\mathbf{h}^i; \mathcal{A}), \quad i = 1, \dots, t. \quad (16)$$

The base learner then follows its original candidate selection, aggregation, or prediction procedure. Thus, every candidate is interpreted through the same accumulated recall space without changing the semantics of the base method.

TAILS is also compatible with different classifier types. For a parametric classifier, such as a linear head, the shaped representation is passed directly to the original classifier:

$$\hat{y} = \arg \max_c G_{\mathcal{B}}(\mathbf{h}')_c. \quad (17)$$

For prototype-based cosine classifiers (Sun et al. 2025; Zhou et al. 2025a), the classifier is parameterized by stored class prototypes. Since TAILS changes the representation used for prediction, each class prototype is transformed by the same unified mapping to preserve sample–prototype consistency:

$$\mu'_c = \mathcal{F}_{\text{TAILS}}(\mu_c; \mathcal{A}). \quad (18)$$

The prediction rule is then:

$$\hat{y} = \arg \max_c \text{sim}_{\cos}(\mathbf{h}', \mu'_c). \quad (19)$$

Consequently, deploying TAILS requires no structural modification to the original method. Its prediction mechanism remains intact, while task-anchored latent recall only reshapes the representation supplied to it.

5 Experiments

This section conducts extensive experiments to evaluate TAILS. Sec. 5.2 compares TAILS with representative PTM-based CL methods across different adaptation paradigms. Sec. 5.3 analyzes the training objectives, underlying mechanism, inference efficiency, and parameter overhead. Sec. 5.4 studies the sensitivity of TAILS to the number of support anchors and task slots. T-SNE visualizations of the feature spaces are provided in the supplementary material.

5.1 Implementation Details

Dataset and splits. We select four benchmark datasets that exhibit notable domain gaps (Zhou et al. 2024b) relative to the ImageNet-21K pretraining distribution, including Stanford Cars (Krause et al. 2013), ImageNet-A (Hendrycks et al. 2021b), ImageNet-R (Hendrycks et al. 2021a), and VTAB (Zhai et al. 2019). Following the “B/Base-m, Inc-n” protocol (Zhou et al. 2025a), datasets are split into CARS B16 Inc20, ImageNet-A B20 Inc20, VTAB B10 Inc10, and ImageNet-R B5 Inc5. ImageNet-R with B5 Inc5 forms a 40-task benchmark, allowing us to evaluate whether TAILS remains effective when task-level ambiguity accumulates over long-sequence CL.

Baselines. We deploy TAILS on baseline or SOTA PTM-based CL approaches that cover different paradigms: prompt-based method DualPrompt (Wang et al. 2022a), raw PTM and prototype classification method SimpleCIL (Zhou et al. 2025a), multi-task adapter methods MOS (Sun et al. 2025) and SEMA (Wang et al. 2025), adapter-based integration method TUNA (Wang, Zhou, and Ye 2025), and task-specific LoRA method CL-LoRA (He, Duan, and Zhu 2025). Following the original settings of each method, DualPrompt and SEMA use ViT-B/16-224, while the remaining methods adopt ViT-B/16-IN21K as the PTM backbone.

Programming and hyperparameters. All experiments are implemented in PyTorch 2.4.1 and conducted on NVIDIA 3090 GPUs. Baseline settings follow the configurations in original papers or LAMDA-Pilot (Sun et al. 2023). For TAILS, the number of task slots K is mainly set to 5, and the number of support anchors for each class is mainly set to 20. TAILS adopts the training configuration of the corresponding base method, including the optimizer, initial learning rate, batch size, number of epochs, and learning-rate schedule.

Evaluation metrics. We use A_T to evaluate the final classification accuracy on the joint test set of all tasks after training on the last task, and use A_T^{task} to measure whether the task of the predicted class matches the ground-truth task.

5.2 Benchmark Comparison

This subsection evaluates TAILS by deploying it on various PTM-based CL methods. Table 1 reports the mean and standard deviation of A_T^{task} and A_T over five seeds on four benchmark datasets. For high-performing baselines, A_T^{task} is often close to A_T , indicating that their within-task prediction is already strong, while cross-task inference becomes the main factor limiting final performance. TAILS effectively improves both cross-task prediction accuracy and final classification accuracy, enabling the enhanced learners to surpass

Method	CARS-196 B16 Inc20		ImageNetA-200 B20 Inc20		ImageNetR-200 B5 Inc5		VTAB-50 B10 Inc10	
	A_T^{task}	A_T	A_T^{task}	A_T	A_T^{task}	A_T	A_T^{task}	A_T
SimpleCIL	43.53 $\pm$ 0.01	37.68 $\pm$ 0.01	52.98 $\pm$ 0.01	48.67 $\pm$ 0.01	55.19 $\pm$ 0.01	54.59 $\pm$ 0.02	87.11 $\pm$ 0.01	84.33 $\pm$ 0.01
SimpleCIL-TAILS	68.03 $\pm$ 0.97	64.39 $\pm$ 0.95	58.70 $\pm$ 0.66	54.86 $\pm$ 0.60	68.69 $\pm$ 0.99	67.94 $\pm$ 0.93	91.98 $\pm$ 0.08	90.18 $\pm$ 0.04
DualPrompt	34.90 $\pm$ 0.92	32.73 $\pm$ 0.96	59.48 $\pm$ 0.80	56.71 $\pm$ 0.84	65.69 $\pm$ 0.94	65.49 $\pm$ 0.98	79.75 $\pm$ 2.84	79.07 $\pm$ 2.90
DualPrompt-TAILS	61.70 $\pm$ 0.95	59.76 $\pm$ 0.93	65.07 $\pm$ 0.83	60.86 $\pm$ 0.81	74.30 $\pm$ 0.95	73.89 $\pm$ 0.97	90.16 $\pm$ 2.85	88.30 $\pm$ 2.89
SEMA	57.05 $\pm$ 0.85	55.54 $\pm$ 0.91	64.82 $\pm$ 0.35	58.62 $\pm$ 0.41	69.31 $\pm$ 0.85	69.11 $\pm$ 0.91	94.93 $\pm$ 0.86	89.51 $\pm$ 0.92
SEMA-TAILS	67.23 $\pm$ 0.90	65.72 $\pm$ 0.86	67.71 $\pm$ 0.40	61.12 $\pm$ 0.36	70.82 $\pm$ 0.86	70.56 $\pm$ 0.90	98.48 $\pm$ 0.88	92.03 $\pm$ 0.90
MOS	73.52 $\pm$ 0.87	70.66 $\pm$ 0.89	58.81 $\pm$ 0.39	55.20 $\pm$ 0.37	71.17 $\pm$ 0.89	70.55 $\pm$ 0.87	99.54 $\pm$ 0.91	92.25 $\pm$ 0.87
MOS-TAILS	75.48 $\pm$ 0.89	72.75 $\pm$ 0.87	60.92 $\pm$ 0.36	57.96 $\pm$ 0.40	73.90 $\pm$ 0.85	73.32 $\pm$ 0.91	99.65 $\pm$ 0.85	92.94 $\pm$ 0.93
TUNA	77.66 $\pm$ 0.90	76.61 $\pm$ 0.86	66.21 $\pm$ 0.37	64.18 $\pm$ 0.39	75.74 $\pm$ 0.87	75.32 $\pm$ 0.89	98.28 $\pm$ 0.90	91.80 $\pm$ 0.88
TUNA-TAILS	79.06 $\pm$ 0.87	78.09 $\pm$ 0.89	67.73 $\pm$ 0.41	65.30 $\pm$ 0.35	77.16 $\pm$ 0.89	76.74 $\pm$ 0.87	98.44 $\pm$ 0.91	92.27 $\pm$ 0.87
CL-LoRA	65.49 $\pm$ 0.51	64.36 $\pm$ 0.55	59.41 $\pm$ 0.12	57.62 $\pm$ 0.16	73.62 $\pm$ 0.61	73.49 $\pm$ 0.65	97.67 $\pm$ 0.32	94.22 $\pm$ 0.36
CL-LoRA-TAILS	70.12 $\pm$ 0.53	69.41 $\pm$ 0.56	61.06 $\pm$ 0.13	59.33 $\pm$ 0.15	74.15 $\pm$ 0.66	74.01 $\pm$ 0.62	98.18 $\pm$ 0.33	94.71 $\pm$ 0.35

Table 1: Comparison of task prediction accuracy A_T^{task} and final classification accuracy A_T . ImageNet-R uses the B5 Inc5 split with 200 classes, resulting in a 40-task long-sequence benchmark. The best results are highlighted in bold.

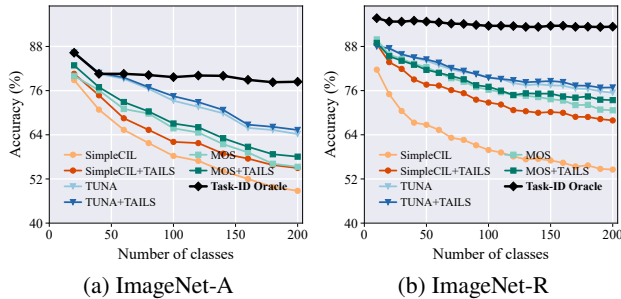


Figure 3: Accuracy trajectories of different methods on ImageNet-A and ImageNet-R.

existing state-of-the-art PTM-based CL methods on multiple benchmarks. The improvements are also consistent on the 40-task ImageNet-R dataset, demonstrating that TAILS remains effective on the long-sequence CL benchmark. These results demonstrate the broad applicability of TAILS to diverse PTM-based CL paradigms. In particular, TAILS still yields notable improvements when applied to strong SOTA baselines such as TUNA and CL-LoRA. We further report the accuracy trajectories throughout continual learning in Fig. 3. TAILS consistently performs best in all evaluated settings, and the gains remain evident as more classes are introduced. The complete curves for all methods and datasets are provided in the supplementary material.

5.3 Ablation Study

This subsection conducts a series of ablation studies on TAILS. We first investigate the effectiveness of the three training objectives in TAILS on the ImageNetA dataset, as shown in Table 2. The results show that $\mathcal{L}_{\text{cls}}$, $\mathcal{L}_{\text{task}}$, and $\mathcal{L}_{\text{stab}}$ all contribute to the final performance. The relative importance of $\mathcal{L}_{\text{cls}}$ and $\mathcal{L}_{\text{task}}$ varies across different base learners.

$\mathcal{L}_{\text{cls}}$	$\mathcal{L}_{\text{task}}$	$\mathcal{L}_{\text{stab}}$	SimpleCIL-TAILS	TUNA-TAILS
×	×	×	48.67	64.18
×	✓	✓	50.51	65.17
✓	×	✓	54.12	64.91
✓	✓	×	54.78	65.02
✓	✓	✓	54.86	65.30

Table 2: Ablation study of the training objectives in TAILS. All results are averaged over five seeds.

Variant	CARS				ImageNet-R			
	SimpleCIL	TUNA	MOS	CL-LoRA	SimpleCIL	TUNA	MOS	CL-LoRA
Anchor-MLP	58.81	77.32	71.34	65.92	62.43	75.91	72.15	73.57
Anchor Retrieval	56.66	77.04	71.03	66.02	60.47	75.51	71.37	73.48
Single-Center	62.23	77.72	70.54	67.52	65.95	76.23	72.55	73.91
Logit Calibration	45.22	76.63	70.81	64.61	56.82	75.33	70.42	73.31
TAILS	64.39	78.09	72.75	69.41	67.94	76.74	73.32	74.01

Table 3: Mechanism comparison on CARS and ImageNet-R. All results are averaged over five seeds. All variants use the same anchor and optimization budgets.

For methods such as SimpleCIL, where classification is not sufficiently optimized during continual learning, $\mathcal{L}_{\text{cls}}$ plays a more important role in directly improving classification accuracy. In contrast, TUNA has already learned task-specific classifiers through task-wise adaptation, so $\mathcal{L}_{\text{task}}$ becomes critical for organizing task-level responses and improving task inference. $\mathcal{L}_{\text{stab}}$ further provides a consistent gain by preventing the shaped representation from drifting excessively away from the original PTM feature.

Moreover, we compare TAILS with four competitive variants under the same anchor and optimization budgets, as shown in Table 3. All results are averaged over five random seeds. For each seed, the same sampled anchors are shared by TAILS and all mechanism variants. Anchor-MLP removes the TAILS architecture and uses anchors only to

Method	ImageNet-A		ImageNet-R	
	A_T	$\bar{t}_{\text{inf}}$ (ms)	A_T	$\bar{t}_{\text{inf}}$ (ms)
SimpleCIL	48.67	4.73	54.59	4.82
SimpleCIL-TAILS	54.86	4.92	67.94	5.10
ACMap	54.49	4.93	69.33	5.05
TUNA	64.18	62.78	75.32	145.04
TUNA-TAILS	65.30	66.09	76.74	150.42

Table 4: Comparison of average final accuracy and the per-sample inference time measured over the test set.

assist the incremental update of an MLP. Anchor Retrieval directly computes attention between the input feature and all stored anchors, and uses their weighted sum for feature shaping. Single-Center just represents each task with one center rather than multiple slots. Logit Calibration applies TAILS to the logits instead of feature shaping. TAILS consistently performs best across all evaluated settings. The consistent gains over Anchor-MLP, Anchor Retrieval, and Single-Center validate the TAILS module design. Direct logit calibration is brittle because even small corrections can change class rankings and destabilize the final prediction. Moreover, cross-task ambiguity is a representation-level problem that is difficult to resolve by modifying the classifier alone.

Furthermore, we investigate the inference-time overhead introduced by TAILS. For a more comprehensive comparison, we include ACMap (Fukuda, Kera, and Kawamoto 2025), a PTM-based method specifically optimized for inference speed as an efficiency-oriented reference. We report the mean final accuracy and the average per-sample inference time on the test set over multiple runs with five seeds, as shown in Table 4. TAILS introduces only marginal inference overhead over the original method. Notably, applying TAILS to SimpleCIL directly yields a learner that is competitive with state-of-the-art efficiency-oriented methods in both accuracy and inference speed.

Methods	Model Params.	Auxiliary Storage	Total Storage
ACMap	1.19	47.59	48.78
MOS	12.17	130.29	142.46
TUNA	12.48	118.12	130.60
CL-LoRA	7.56	0.00	7.56
TAILS	1.14	3.07	4.21

Table 5: Comparison of additional storage overhead on the ImageNet-R dataset, measured in millions of scalar values (M), excluding the PTM.

In addition, Table 5 compares the additional storage required by different methods under the ImageNet-R protocol. For a consistent comparison, we count both the model parameters and all auxiliary tensors that must be retained for subsequent learning or inference. The auxiliary storage includes the task anchors maintained by TAILS, the class-alignment statistics used by MOS and TUNA, and the task-specific adapter checkpoints retained by ACMap. For CL-LoRA, all task-expanded LoRA weights are included in the model parameters, with no separately stored auxiliary tensors. TAILS

is lightweight on its own, and its plug-in overhead remains below the learner-specific storage introduced by every compared method. Relative to the storage of the corresponding base learner, adding TAILS incurs only 3.0% overhead for MOS, 3.2% for TUNA, and 8.6% for ACMap; even for the more compact CL-LoRA, the additional storage remains below its original task-expanded parameter footprint.

5.4 Sensitivity Analysis of Hyperparameters

We study how key hyperparameters in TAILS affect performance, with the results shown in Fig. 4. The results indicate that even when no support anchors are used, TAILS can still improve the base learner by relying solely on class prototypes as task anchors. Increasing the number of support anchors generally improves final classification accuracy, as additional anchors capture more within-class variation and support richer task-slot representations for composing latent recall. However, the gains diminish as additional anchors become increasingly redundant. Increasing the number of task slots further improves performance by capturing finer structures within each task. Too many slots, however, can fragment the task representation and make query-slot matching less reliable, thereby degrading latent-recall composition.

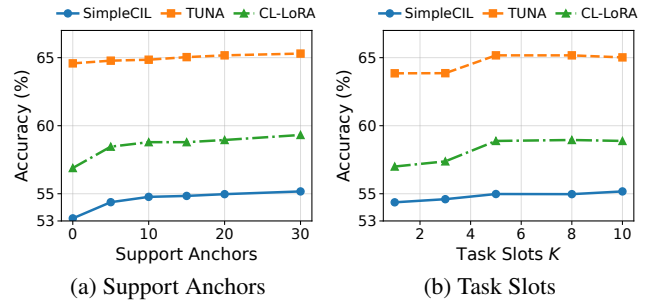


Figure 4: Impact of support anchor and task slot counts on the results across different PTM-based CL methods.

6 Conclusion

This paper introduces TAILS, a task-anchored latent-recall-assisted inference module for PTM-based continual learning. TAILS interprets each current representation against fixed accumulated task structures, composes input-conditioned latent recall, and internalizes the recalled evidence as a recall-conditioned feature correction. It is shared across all learned tasks and incrementally updated in a decoupled manner, so accumulated task-level information can refine current representations while preserving the original training pipeline of the base learner. We further store class prototypes and support features as fixed task anchors in the feature space, providing stable references for training and inference. Extensive experiments across various continual learning methods demonstrate that TAILS consistently improves baseline performance with modest parameter overhead and negligible inference cost. Future work will explore more expressive anchor construction strategies and stronger shaping objectives to further enhance task-level discriminability.

References

- Aghasanli, A.; Li, Y.; and Angelov, P. 2025. Prototype-Based Continual Learning with Label-free Replay Buffer and Cluster Preservation Loss. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, 6545–6554.
- Ba, J. L.; Kiros, J. R.; and Hinton, G. E. 2016. Layer Normalization. *arXiv preprint arXiv:1607.06450*.
- Chen, H.; Wang, P.; Zhou, Z.; Zhang, X.; Wu, Z.; and Jiang, Y.-G. 2025. Achieving More with Less: Additive Prompt Tuning for Rehearsal-Free Class-Incremental Learning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 340–349.
- Chen, S.; Gong, C.; Li, J.; Yang, J.; Niu, G.; and Sugiyama, M. 2022. Learning contrastive embedding in low-dimensional space. *Advances in Neural Information Processing Systems*, 35: 6345–6357.
- Chen, S.; Niu, G.; Gong, C.; Li, J.; Yang, J.; and Sugiyama, M. 2021. Large-margin contrastive learning with distance polarization regularizer. In *International Conference on Machine Learning*, 1673–1683. PMLR.
- De Lange, M.; Aljundi, R.; Masana, M.; Parisot, S.; Jia, X.; Leonardis, A.; Slabaugh, G.; and Tuytelaars, T. 2021. A continual learning survey: Defying forgetting in classification tasks. *IEEE transactions on pattern analysis and machine intelligence*, 44(7): 3366–3385.
- Fukuda, T.; Kera, H.; and Kawamoto, K. 2025. Adapter Merging with Centroid Prototype Mapping for Scalable Class-Incremental Learning. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, 4884–4893.
- Gao, R.; Zhao, Q.; and Fu, K. 2026. DiAPR: Dimensionally-Allocated Prototype Refinement for Non-Exemplar Class Incremental Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 21189–21197.
- Gao, Z.; Han, S.; Zhang, X.; Xu, K.; Zhou, D.; Mao, X.; Dou, Y.; and Wang, H. 2025. Maintaining fairness in logit-based knowledge distillation for class-incremental learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, 16763–16771.
- Gomes, H. M.; Barddal, J. P.; Enembreck, F.; and Bifet, A. 2017. A survey on ensemble learning for data stream classification. *ACM Computing Surveys (CSUR)*, 50(2): 1–36.
- He, J.; Duan, Z.; and Zhu, F. 2025. CL-LoRA: Continual Low-Rank Adaptation for Rehearsal-Free Class-Incremental Learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 30534–30544.
- Hendrycks, D.; Basart, S.; Mu, N.; Kadavath, S.; Wang, F.; Dorundo, E.; Desai, R.; Zhu, T.; Parajuli, S.; Guo, M.; et al. 2021a. The many faces of robustness: A critical analysis of out-of-distribution generalization. In *Proceedings of the IEEE/CVF international conference on computer vision*, 8340–8349.
- Hendrycks, D.; Zhao, K.; Basart, S.; Steinhardt, J.; and Song, D. 2021b. Natural adversarial examples. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 15262–15271.
- Houlsby, N.; Giurgiu, A.; Jastrzebski, S.; Morrone, B.; De Laroussilhe, Q.; Gesmundo, A.; Attariyan, M.; and Gelly, S. 2019. Parameter-efficient transfer learning for NLP. In *International conference on machine learning*, 2790–2799. PMLR.
- Krause, J.; Stark, M.; Deng, J.; and Fei-Fei, L. 2013. 3D Object Representations for Fine-Grained Categorization. In *Proceedings of the IEEE International Conference on Computer Vision Workshops*, 554–561.
- Lai, G.; Li, Y.; Wang, X.; Zhang, J.; Li, T.; and Yang, X. 2025. Order-Robust Class Incremental Learning: Graph-Driven Dynamic Similarity Grouping. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Lee, J.; Hayat, M.; and Yun, S. 2025. Tripartite Weight-Space Ensemble for Few-Shot Class-Incremental Learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 15329–15338.
- Li, Y.; Wang, H.; Qi, Y.; Liu, W.; and Li, R. 2025. Re-fed+: A better replay strategy for federated incremental learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
- Liang, Y.-S.; and Li, W.-J. 2024. Loss decoupling for task-agnostic continual learning. *Advances in Neural Information Processing Systems*, 36: 11151–11167.
- Liu, S.; et al. 2025. Enhancing Online Continual Learning with Plug-and-Play State Space Model and Class-Conditional Mixture of Discretization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Nori, M. K.; Kim, I.-M.; and Wang, G. 2025. Federated class-incremental learning: A hybrid approach using latent exemplars and data-free techniques to address local and global forgetting. *arXiv preprint arXiv:2501.15356*.
- Smith, J. S.; Karlinsky, L.; Gutta, V.; Cascante-Bonilla, P.; Kim, D.; Arbelle, A.; Panda, R.; Feris, R.; and Kira, Z. 2023. Coda-prompt: Continual decomposed attention-based prompting for rehearsal-free continual learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 11909–11919.
- Sun, H.-L.; Zhou, D.-W.; Ye, H.-J.; and Zhan, D.-C. 2023. Pilot: A pre-trained model-based continual learning toolbox. *arXiv preprint arXiv:2309.07117*.
- Sun, H.-L.; Zhou, D.-W.; Zhao, H.; Gan, L.; Zhan, D.-C.; and Ye, H.-J. 2025. Mos: Model surgery for pre-trained model-based class-incremental learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, 20699–20707.
- Wang, H.; Lu, H.; Yao, L.; and Gong, D. 2025. Self-Expansion of Pre-trained Models with Mixture of Adapters for Continual Learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 10087–10098.

- Wang, Y.; Zhou, D.-W.; and Ye, H.-J. 2025. Integrating Task-Specific and Universal Adapters for Pre-Trained Model-based Class-Incremental Learning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 806–816.
- Wang, Z.; Zhang, Z.; Ebrahimi, S.; Sun, R.; Zhang, H.; Lee, C.-Y.; Ren, X.; Su, G.; Perot, V.; Dy, J.; et al. 2022a. Dual-prompt: Complementary prompting for rehearsal-free continual learning. In *European Conference on Computer Vision*, 631–648. Springer.
- Wang, Z.; Zhang, Z.; Lee, C.-Y.; Zhang, H.; Sun, R.; Ren, X.; Su, G.; Perot, V.; Dy, J.; and Pfister, T. 2022b. Learning to prompt for continual learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 139–149.
- Wu, Y.; Piao, H.; Huang, L.-K.; Wang, R.; Li, W.; Pfister, H.; Meng, D.; Ma, K.; and Wei, Y. 2025. SD-LoRA: Scalable Decoupled Low-Rank Adaptation for Class Incremental Learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- Xu, Z.; Yang, S.; Xu, B.; Shen, F.; and Zhao, J. 2025. Dual Prototypes for Adaptive Pre-Trained Model in Class-Incremental Learning. *Neural Networks*, 108389.
- Yang, S.; Shen, F.; and Zhao, J. 2024. EntAugment: Entropy-Driven Adaptive Data Augmentation Framework for Image Classification. In *European Conference on Computer Vision*, 197–214. Springer.
- Yang, S.; Zhang, T.; Xu, Z.; Li, P.; Xu, B.; Shen, F.; and Zhao, J. 2025. Supervised contrastive learning with prototype distillation for data incremental learning. *Neural Networks*, 107651.
- Ye, H.-J.; Zhan, D.-C.; Li, N.; and Jiang, Y. 2019. Learning multiple local metrics: Global consideration helps. *IEEE transactions on pattern analysis and machine intelligence*, 42(7): 1698–1712.
- Yi, H.; Xu, Z.; Tu, D.; Wang, Z.; Xu, B.; and Shen, F. 2026. Beyond Point-wise Neural Collapse: A Topology-Aware Hierarchical Classifier for Class-Incremental Learning. *arXiv preprint arXiv:2605.11904*.
- Zang, H.; Dong, Y.; Li, L.; Yang, L.; and Wang, Y. 2026. Topology-aware Knowledge Preservation for Class-Incremental Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- Zhai, X.; Puigcerver, J.; Kolesnikov, A.; Ruysen, P.; Riquelme, C.; Lucic, M.; Djolonga, J.; Pinto, A. S.; Neumann, M.; Dosovitskiy, A.; et al. 2019. A large-scale study of representation learning with the visual task adaptation benchmark. *arXiv preprint arXiv:1910.04867*.
- Zhang, S.; Lv, X.; Xing, Y.; Wu, Q.; Xu, D.; and Zhang, Y. 2025. Revisiting Generative Replay for Class Incremental Object Detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 20340–20349.
- Zheng, B.; Zhou, D.-W.; Ye, H.-J.; and Zhan, D.-C. 2025. Task-Agnostic Guided Feature Expansion for Class-Incremental Learning. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, 10099–10109.
- Zhou, D.-W.; Cai, Z.-W.; Ye, H.-J.; Zhan, D.-C.; and Liu, Z. 2025a. Revisiting Class-Incremental Learning with Pre-Trained Models: Generalizability and Adaptivity Are All You Need. *International Journal of Computer Vision*.
- Zhou, D.-W.; Sun, H.-L.; Ning, J.; Ye, H.-J.; and Zhan, D.-C. 2024a. Continual learning with pre-trained models: A survey. *arXiv preprint arXiv:2401.16386*.
- Zhou, D.-W.; Sun, H.-L.; Ye, H.-J.; and Zhan, D.-C. 2024b. Expandable Subspace Ensemble for Pre-Trained Model-Based Class-Incremental Learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 23554–23564.
- Zhou, D.-W.; Wang, Q.-W.; Qi, Z.-H.; Ye, H.-J.; Zhan, D.-C.; and Liu, Z. 2024c. Class-incremental learning: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
- Zhou, D.-W.; et al. 2025b. Dual Consolidation for Pre-Trained Model-Based Domain-Incremental Learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Zhu, G.; Wu, D.; Gao, C.; Wang, R.; Yang, W.; and Sang, N. 2025. Adaptive Prototype Replay for Class Incremental Semantic Segmentation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, 10932–10940.